

TECNICATURA UNIVERSITARIA EN PROGRAMACIÓN A DISTANCIA
Universidad Tecnológica Nacional

Trabajo Práctico Integrador

Gestión de Datos de Países en Python:
filtros, ordenamientos y estadísticas

Materia: Programación 1
Estudiante: Claudia Eugenia González
Cohorte: Marzo 2026
Fecha de entrega: 15/ 06/ 2026

Repositorio del proyecto:
<https://github.com/ceugeniag/tpi-programacion1-paises>
Video demostrativo:
https://youtu.be/V_z6sULx84s

Índice

1. Marco Teórico	3
1.1 Listas	3
1.2 Diccionarios	3
1.3 Funciones	3
1.4 Estructuras Condicionales y Repetitivas	3
1.5 Ordenamientos	4
1.6 Estadísticas Básicas	4
1.7 Archivos CSV	4
2. Decisiones Técnicas y Arquitectura	5
2.1 Estructura de Datos Elegida	5
2.2 Diagrama de Flujo	5
2.3 Capturas de Ejecución	7
3. Dificultades y Conclusiones	12
4. Bibliografía / Webgrafía	13
5. Enlaces del Proyecto	13

1. Marco Teórico

A continuación se explican los conceptos fundamentales de Programación 1 aplicados en el desarrollo del sistema de gestión de datos de países, relacionando cada uno con su implementación concreta dentro del proyecto.

1.1 Listas

Una lista en Python es una estructura de datos ordenada y mutable que permite almacenar una colección de elementos. En este proyecto, la lista `países` constituye la estructura principal del sistema: cada elemento de la lista es un diccionario que representa un país. Las listas permiten recorrer, filtrar y ordenar la información de manera sencilla mediante bucles y funciones como `sorted()`, y comprensión de listas (*list comprehension*), utilizada extensamente en las funciones de filtrado del programa (por ejemplo, `filtrar_por_continente`).

1.2 Diccionarios

Un diccionario es una colección de pares clave-valor, ideal para representar entidades con múltiples atributos. Cada país del sistema se modela como un diccionario con las claves `nombre`, `poblacion`, `superficie` y `continente`. Esta estructura permite acceder a cada dato de forma directa y legible (`pais['poblacion']`), y resulta coherente con el formato de filas del archivo CSV, lo que facilita la lectura y escritura de datos.

1.3 Funciones

Las funciones permiten dividir un programa en bloques reutilizables, cada uno con una responsabilidad específica (principio de modularización). En este proyecto se definieron funciones independientes para cada operación: carga y guardado de datos (`cargar_paises`, `guardar_paises`), altas y actualizaciones (`agregar_pais`, `actualizar_pais`), búsquedas y filtros, ordenamientos y cálculo de estadísticas. Esta organización mejora la legibilidad, facilita la detección de errores y permite probar cada función de manera aislada.

1.4 Estructuras Condicionales y Repetitivas

Las estructuras condicionales (`if`, `elif`, `else`) se utilizan para validar datos ingresados por el usuario (por ejemplo, verificar que un nombre no esté vacío o que un número sea positivo) y para controlar el flujo del menú principal según la opción seleccionada. Las estructuras repetitivas (`while` y `for`) se emplean para mantener el menú interactivo activo hasta que el usuario decide salir, para insistir en la carga de datos válidos, y para recorrer la lista de países al mostrar tablas o calcular estadísticas.

1.5 Ordenamientos

El ordenamiento de datos permite reorganizar una colección según un criterio determinado. En Python, la función `sorted()` permite ordenar una lista a partir de una clave personalizada mediante el parámetro `key`, que en este proyecto se implementa con funciones `lambda` (por ejemplo, `key=lambda p: p['poblacion']`). El parámetro `reverse=True` permite invertir el orden, lo que se utiliza para ofrecer ordenamientos ascendentes y descendentes por nombre, población o superficie.

1.6 Estadísticas Básicas

El sistema calcula indicadores estadísticos simples a partir del dataset: el país con mayor y menor población (mediante las funciones `max()` y `min()` con clave personalizada), el promedio de población y superficie (sumando los valores con `sum()` y dividiendo por la cantidad de elementos), y la cantidad de países por continente, obtenida acumulando conteos en un diccionario auxiliar. Estas estadísticas permiten obtener una visión general del dataset sin necesidad de recorrerlo manualmente.

1.7 Archivos CSV

CSV (*Comma-Separated Values*) es un formato de texto plano utilizado para almacenar datos tabulares, donde cada línea representa un registro y los valores se separan por comas. Python ofrece el módulo estándar `csv`, que permite leer y escribir este tipo de archivos de forma sencilla. En este proyecto se utiliza `csv.DictReader` para leer cada fila del archivo `países.csv` directamente como un diccionario, y `csv.DictWriter` para volcar la lista actualizada de países al archivo. La lectura está protegida con bloques `try/except` para manejar errores de archivo inexistente (`FileNotFoundError`) o de conversión de tipos (`ValueError`).

2. Decisiones Técnicas y Arquitectura

2.1 Estructura de Datos Elegida

Se optó por representar el dataset completo como una **lista de diccionarios**, donde cada diccionario corresponde a un país con las claves `nombre`, `poblacion`, `superficie` y `continente`. Esta decisión se tomó porque:

- Es la representación más directa del formato CSV original, fila por fila.
- Permite acceder a cada atributo por su nombre (`pais['poblacion']`), mejorando la legibilidad del código frente a listas paralelas o tuplas posicionales.
- Es compatible de forma nativa con `csv.DictReader` y `csv.DictWriter`, simplificando la carga y el guardado de datos.
- Facilita el uso de funciones de orden superior como `sorted()`, `filter()` (mediante comprensión de listas), `max()` y `min()` con clave personalizada.

2.2 Diagrama de Flujo

El siguiente diagrama representa el flujo principal de operaciones del sistema: la carga inicial de datos, la presentación del menú interactivo, y el conjunto de operaciones agrupadas según su finalidad (consultas, modificaciones, y ordenamiento/persistencia), todas ellas retornando al menú principal hasta que el usuario selecciona la opción de salir.

Diagrama de Flujo - Gestión de Datos de Países

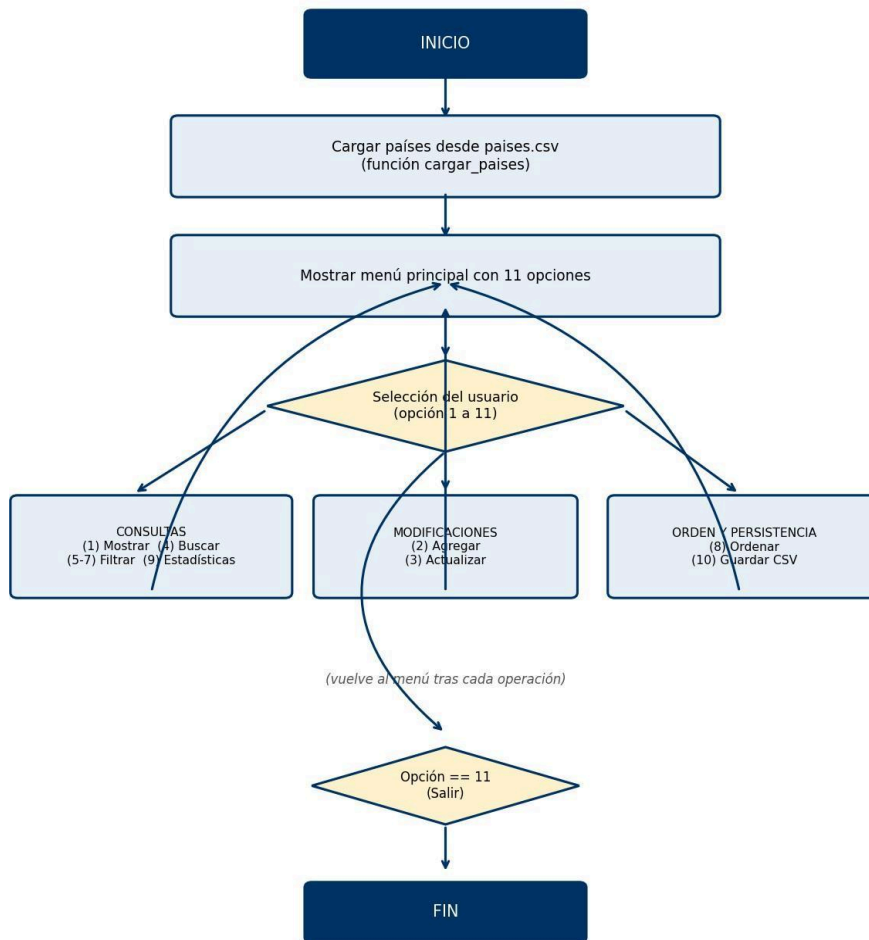


Figura 1. Diagrama de flujo general del sistema.

2.3 Capturas de Ejecución

A continuación se muestran capturas de pantalla de la ejecución del programa en consola, ilustrando distintas funcionalidades del sistema.

```
Terminal - Python

[OK] Se cargaron 20 países desde 'países.csv'.

=====
GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir

=====
Seleccione una opción:
Nombre                Población    Superficie (km²) Continente
-----
Argentina             45,376,763   2,780,400 América
Japón                 125,800,000   377,975 Asia
Brasil                213,993,437   8,515,767 América
Alemania              83,149,300    357,022 Europa
Francia               67,750,000    551,695 Europa
Canadá                38,250,000    9,984,670 América
Egipto                104,258,327   1,010,408 África
Sudáfrica             60,142,978    1,221,037 África
Australia             25,690,000    7,692,024 Oceanía
India                 1,380,004,385 3,287,263 Asia
China                 1,412,600,000 9,596,960 Asia
México                128,972,439   1,964,375 América
Italia                59,236,213    301,340 Europa
España                47,351,567    505,990 Europa
Nigeria              206,139,589    923,768 África
Rusia                 146,171,015   17,098,242 Europa
Estados Unidos        331,002,651   9,833,517 América
Chile                 19,116,209    756,102 América
Corea del Sur          51,780,579    100,210 Asia
Nueva Zelanda          5,084,300     270,467 Oceanía

=====
GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir

=====
Seleccione una opción:
>> Saliendo del sistema. ¡Hasta pronto!
```

Figura 2. Opción 1: listado completo de países cargados desde el CSV.

```
Terminal - Python

[OK] Se cargaron 20 países desde 'países.csv'.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir
=====
Seleccione una opción:
=====
ESTADÍSTICAS GENERALES
=====
País con MAYOR población: China (1,412,600,000 hab.)
País con MENOR población: Nueva Zelanda (5,084,300 hab.)

Promedio de población: 227,593,487.60 hab.
Promedio de superficie: 3,856,461.60 km²

Cantidad de países por continente:
- América: 6
- Asia: 4
- Europa: 5
- África: 3
- Oceanía: 2
=====

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir
=====
Seleccione una opción:
>> Saliendo del sistema. ¡Hasta pronto!
```

Figura 3. Opción 9: estadísticas generales (mayor/menor población, promedios y conteo por continente).

```
Terminal - Python

[OK] Se cargaron 20 países desde 'países.csv'.

=====
GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir

=====
Seleccione una opción:
Ingrese el continente a filtrar:
[OK] Países en Europa:

Nombre                Población    Superficie (km²) Continente
-----
Alemania              83,149,300    357,022 Europa
Francia              67,750,000    551,695 Europa
Italia               59,236,213    301,340 Europa
España              47,351,567    505,990 Europa
Rusia               146,171,015    17,098,242 Europa

=====
GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir

=====
Seleccione una opción:
>> Saliendo del sistema. ¡Hasta pronto!
```

Figura 4. Opción 5: filtro de países por continente (América).

```
Terminal - Python

[OK] Se cargaron 20 países desde 'países.csv'.

=====
GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir

=====
Seleccione una opción:
--- Ordenar países por ---
1. Nombre
2. Población
3. Superficie
Seleccione un criterio (1-3): ¿Orden ascendente o descendente? (a/d):
[OK] Países ordenados:

Nombre                Población    Superficie (km²) Continente
-----
China                 1,412,600,000    9,596,960 Asia
India                 1,380,004,385    3,287,263 Asia
Estados Unidos        331,002,651      9,833,517 América
Brasil               213,993,437      8,515,767 América
Nigeria              206,139,589      923,768 África
Rusia                146,171,015      17,098,242 Europa
México               128,972,439      1,964,375 América
Japón                125,800,000      377,975 Asia
Egipto              104,258,327      1,010,408 África
Alemania             83,149,300       357,022 Europa
Francia              67,750,000       551,695 Europa
Sudáfrica            60,142,978       1,221,037 África
Italia               59,236,213       301,340 Europa
Corea del Sur        51,780,579       100,210 Asia
España              47,351,567       505,990 Europa
Argentina            45,376,763       2,780,400 América
Canadá               38,250,000       9,984,670 América
Australia            25,690,000       7,692,024 Oceanía
Chile                19,116,209       756,102 América
Nueva Zelanda        5,084,300        270,467 Oceanía

=====
GESTIÓN DE DATOS DE PAÍSES
=====

1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir

=====
Seleccione una opción:
>> Saliendo del sistema. ¡Hasta pronto!
```

Figura 5. Opción 8: ordenamiento de países por población, en orden descendente.

```
Terminal - Python

[OK] Se cargaron 20 países desde 'países.csv'.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir
=====
Seleccione una opción:
--- Agregar nuevo país ---
Nombre del país: [ERROR] El nombre no puede estar vacío.
Nombre del país: Población: Superficie (km²): Continente: [OK] País 'Portugal' agregado correctamente.

=====
GESTIÓN DE DATOS DE PAÍSES
=====
1. Mostrar todos los países
2. Agregar país
3. Actualizar población/superficie de un país
4. Buscar país por nombre
5. Filtrar por continente
6. Filtrar por rango de población
7. Filtrar por rango de superficie
8. Ordenar países
9. Mostrar estadísticas
10. Guardar cambios en el archivo CSV
11. Salir
=====
Seleccione una opción:
>> Saliendo del sistema. ¡Hasta pronto!
```

Figura 6. Opción 2: validación de datos al agregar un país. El sistema detecta el nombre vacío, informa el error y vuelve a solicitar el dato.

3. Dificultades y Conclusiones

Dificultades encontradas

- **Validación de datos numéricos:** al solicitar población y superficie por teclado, el usuario puede ingresar texto en lugar de números. Se resolvió utilizando bloques `try/except ValueError` en cada función que requiere conversión de tipos, junto con bucles `while` que insisten hasta recibir un valor válido.
- **Comparación de nombres de países:** para evitar duplicados o no encontrar un país por diferencias de mayúsculas o espacios, se implementó una función auxiliar que normaliza el texto (`.strip().lower()`) antes de comparar.
- **Actualización parcial de datos:** la opción de actualizar población y/o superficie debía permitir modificar solo uno de los dos campos. Se resolvió permitiendo dejar el campo vacío (presionar Enter) para conservar el valor anterior, validando igualmente que, si se ingresa un valor, este sea numérico y positivo.
- **Formato de tablas en consola:** para que la información se visualice de forma ordenada se utilizó formateo de cadenas con ancho fijo (`f-strings` con especificadores como `{valor:>15,}`), lo que además permite mostrar los números de población con separador de miles.

Conclusiones

El desarrollo de este trabajo permitió aplicar de forma integrada los conceptos centrales de Programación 1. La elección de una lista de diccionarios como estructura de datos demostró ser adecuada para representar entidades con múltiples atributos y resultó totalmente compatible con la lectura y escritura de archivos CSV mediante el módulo estándar `csv`.

La modularización del código en funciones independientes, cada una con una responsabilidad clara, facilitó tanto el desarrollo incremental como la detección y corrección de errores. Asimismo, la incorporación sistemática de validaciones con `try/except` y bucles de reintento mejoró notablemente la robustez del programa frente a entradas inválidas del usuario.

Como aprendizaje principal, se destaca la importancia de pensar el diseño de las estructuras de datos antes de comenzar a programar, ya que una buena elección inicial (en este caso, la lista de diccionarios) simplifica considerablemente la implementación de funcionalidades más avanzadas como filtros combinados, ordenamientos por múltiples criterios y cálculo de estadísticas agregadas.

4. Bibliografía / Webgrafía

- Python Software Foundation. (2025). *The Python Tutorial — Data Structures*. Documentación oficial de Python 3. <https://docs.python.org/3/tutorial/datastructures.html>
- Python Software Foundation. (2025). *csv — CSV File Reading and Writing*. Documentación oficial de Python 3. <https://docs.python.org/3/library/csv.html>
- Python Software Foundation. (2025). *Built-in Functions: sorted(), max(), min()*. Documentación oficial de Python 3. <https://docs.python.org/3/library/functions.html>
- Python Software Foundation. (2025). *Errors and Exceptions*. Documentación oficial de Python 3. <https://docs.python.org/3/tutorial/errors.html>

5. Enlaces del Proyecto

- **Repositorio GitHub:** <https://github.com/ceugeniag/tpi-programacion1-paises>
- **Video demostrativo:** https://youtu.be/V_z6sULx84s
- **Documento PDF:** incluido en la raíz del repositorio (este mismo archivo).